Exercício 8.2: CronJob com concorrência controlada e Secret de duas formas

Aluno: Luiz Brito · **Data:** 24/10/2026 · **Disciplina:** Cloud Native Development (CP3)

Ambiente

- Cluster kind com 3 nós, Kubernetes v1.36.1, namespace `store-dev`.
- Execução automatizada por `04-commands.sh`: 12 min com `Allow`, depois 12 min com `Forbid`, com uma amostra de `kubectl get cronjob,jobs,pods` a cada 30 s (47 amostras). O CronJob foi aplicado às 00:41:11. Ele foi trocado para `Forbid` às 00:53:15 e o script terminou às 01:05.

Manifestos

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: catalogo-secret
  namespace: store-dev
type: Opaque
stringData:
  API_TOKEN: token-v1
```

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: sincroniza-catalogo
  namespace: store-dev
spec:
  schedule: "*/5 * * * *"
  timeZone: "America/Sao_Paulo"
  startingDeadlineSeconds: 120
  concurrencyPolicy: Allow
  successfulJobsHistoryLimit: 2
  failedJobsHistoryLimit: 1
```

```yaml
jobTemplate:
  spec:
    template:
      spec:
        restartPolicy: Never
        containers:
          - name: sincroniza
            image: busybox:1.37
            command: ["sh", "-c", "echo sincronizando catalogo; sleep 360; echo fim"]
            env:
              - name: API_TOKEN
                valueFrom:
                  secretKeyRef:
                    name: catalogo-secret
                    key: API_TOKEN
            volumeMounts:
              - name: token
                mountPath: /etc/catalogo
                readOnly: true
        volumes:
          - name: token
            secret:
              secretName: catalogo-secret
              defaultMode: 0400
              items:
                - key: API_TOKEN
                  path: token
```

`03-sincroniza-cronjob-forbid.yaml` é idêntico, trocando só `concurrencyPolicy: Allow` por `concurrencyPolicy: Forbid`.

Correção durante a execução: a primeira tentativa montava o Secret sem `items`, e o arquivo apareceu como `/etc/catalogo/API_TOKEN`, porque o volume de Secret cria um arquivo por chave, com o nome da chave. O `items` (`key: API_TOKEN`, `path: token`) faz o arquivo se chamar `/etc/catalogo/token`, como pede o enunciado.

Cenário Allow

```
### 00:50:30 ativos=2
NAME                                SCHEDULE      TIMEZONE           SUSPEND   ACTIVE   LAST
SCHEDULE   AGE
cronjob.batch/sincroniza-catalogo   */5 * * * *   America/Sao_Paulo   False     2        30s
9m19s

NAME                                    STATUS    COMPLETIONS   DURATION   AGE
job.batch/sincroniza-catalogo-29880225   Running   0/1           5m30s      5m30s
job.batch/sincroniza-catalogo-29880230   Running   0/1           30s        30s

NAME                                    READY   STATUS    RESTARTS   AGE
pod/sincroniza-catalogo-29880225-kjvd7   1/1     Running   0          5m30s
pod/sincroniza-catalogo-29880230-g2ctz   1/1     Running   0          30s
```

O Job das 00:45 dura 6 min e ainda estava rodando quando o das 00:50 começou: **2 Jobs ativos ao mesmo tempo** entre 00:50:00 e 00:51:00 (3 amostras seguidas com `ativos=2`). O sufixo do Job (`29880230`) é o horário agendado em minutos desde a época Unix, por isso dois Jobs nunca têm o mesmo nome.

Cenário Forbid

```
### 00:56:32 ativos=1
NAME                                SCHEDULE      TIMEZONE           SUSPEND   ACTIVE   LAST
SCHEDULE   AGE
cronjob.batch/sincroniza-catalogo   */5 * * * *   America/Sao_Paulo   False     1        92s
15m

NAME                                    STATUS     COMPLETIONS   DURATION   AGE
job.batch/sincroniza-catalogo-29880225   Complete   1/1           6m3s       11m
job.batch/sincroniza-catalogo-29880230   Complete   1/1           6m3s       6m32s
job.batch/sincroniza-catalogo-29880235   Running    0/1           29s        29s

### 01:00:33 ativos=1
NAME                                SCHEDULE      TIMEZONE           SUSPEND   ACTIVE   LAST
SCHEDULE   AGE
cronjob.batch/sincroniza-catalogo   */5 * * * *   America/Sao_Paulo   False     1        5m33s
19m

NAME                                    STATUS     COMPLETIONS   DURATION   AGE
job.batch/sincroniza-catalogo-29880225   Complete   1/1           6m3s       15m
job.batch/sincroniza-catalogo-29880230   Complete   1/1           6m3s       10m
job.batch/sincroniza-catalogo-29880235   Running    0/1           4m30s      4m30s
```

Nas 24 amostras do cenário `Forbid`, **nenhuma passou de 1 ativo.**

```
$ kubectl -n store-dev describe cronjob sincroniza-catalogo
Name:                          sincroniza-catalogo
Schedule:                      */5 * * * *
Concurrency Policy:            Forbid
Time Zone:                     America/Sao_Paulo
Successful Job History Limit:  2
Failed Job History Limit:      1
Starting Deadline Seconds:     120s
...
Events:
  Type    Reason            Age                    From                Message
  ----    ------            ----                   ----                -------
  Normal  SuccessfulCreate  20m                    cronjob-controller  Created job sincroniza-
catalogo-29880225
  Normal  SuccessfulCreate  15m                    cronjob-controller  Created job sincroniza-
catalogo-29880230
  Normal  SawCompletedJob   14m                    cronjob-controller  Saw completed job: sincroniza-
catalogo-29880225, condition: Complete
  Normal  SawCompletedJob   9m32s (x2 over 9m32s)  cronjob-controller  Saw completed job: sincroniza-
catalogo-29880230, condition: Complete
  Normal  SuccessfulCreate  9m32s                  cronjob-controller  Created job sincroniza-
catalogo-29880235
  Normal  JobAlreadyActive  5m35s (x4 over 10m)    cronjob-controller  Not starting job because prior
execution is running and concurrency policy is Forbid
  Normal  SuccessfulDelete  3m29s                  cronjob-controller  Deleted job sincroniza-
catalogo-29880225
  Normal  SawCompletedJob   3m29s                  cronjob-controller  Saw completed job: sincroniza-
catalogo-29880235, condition: Complete
  Normal  SuccessfulCreate  35s                    cronjob-controller  Created job sincroniza-
catalogo-29880245
```

O evento `JobAlreadyActive` aparece 4 vezes, mas as execuções puladas são duas, com destinos diferentes:

- **00:55: atrasada, não perdida.** O Job das 00:50 ainda rodava, então o controller registrou `JobAlreadyActive` e tentou de novo. O Job das 00:50 terminou às 00:56:03, ainda dentro dos 120 s de `startingDeadlineSeconds`, e o controller criou o `29880235` (o horário das 00:55) no mesmo segundo, com 63 s de atraso (`startTime` 03:56:03Z). Na amostra das 00:56:32, o `LAST SCHEDULE` é `92s`, contado a partir do horário agendado (00:55).
- **01:00: perdida de vez.** O `29880235` só terminou às 01:02:06, depois do prazo de 01:02:00. A execução das 01:00 nunca foi criada. Pela sequência de Jobs (`...230`, `...235`, `...245`), falta o `...240`.

Ou seja, com `Forbid` a execução seguinte **espera** enquanto `startingDeadlineSeconds` permitir e só é perdida depois disso. O "pular" do enunciado depende dos dois parâmetros juntos.

Histórico de Jobs

Contagem de Jobs `Complete` presentes em cada uma das 47 amostras:

```
    19 amostras com 0
    10 amostras com 1
    18 amostras com 2
```

O histórico **nunca passou de 2** concluídos. Quando o terceiro (`29880235`) terminou, o controller apagou o mais antigo (`SuccessfulDelete ... 29880225`).

Secret por env e por volume no mesmo Pod

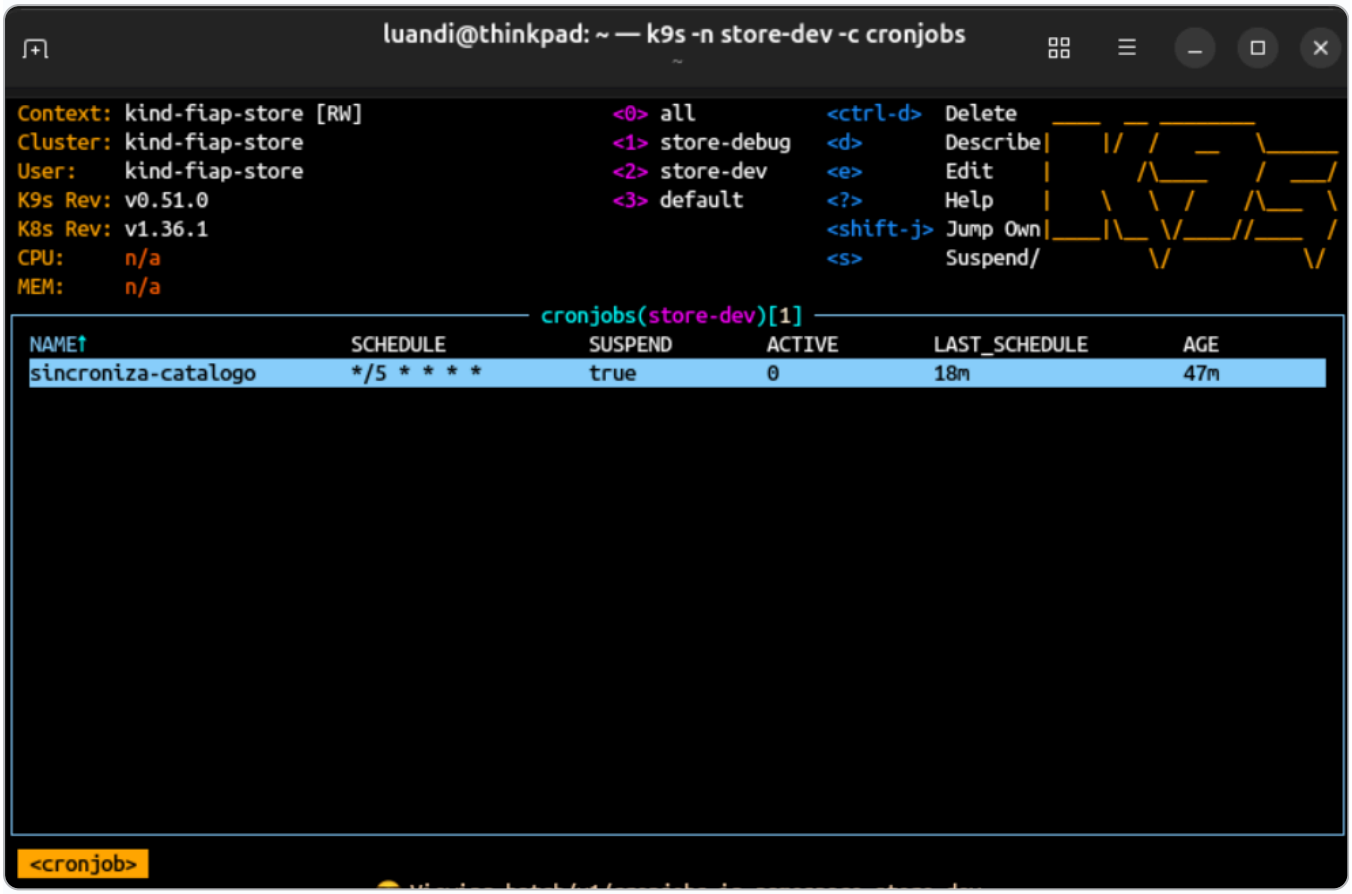
```
### secret antes (pod/sincroniza-catalogo-29880225-kjvd7)   00:45:13
$ kubectl exec <pod> -- printenv API_TOKEN
token-v1
$ kubectl exec <pod> -- cat /etc/catalogo/token
token-v1

$ kubectl -n store-dev patch secret catalogo-secret -p '{"stringData":{"API_TOKEN":"token-v2"}}'
secret/catalogo-secret patched                              (managedFields: 03:45:13Z)

### 75 s depois, primeira checagem do script
$ kubectl exec <pod> -- printenv API_TOKEN
token-v1
$ kubectl exec <pod> -- cat /etc/catalogo/token
token-v1

### 00:47:17, mesmo Pod
$ kubectl exec <pod> -- sh -c 'printenv API_TOKEN; cat /etc/catalogo/token; echo; ls -la
/etc/catalogo/'
token-v1
token-v2
drwxrwxrwt    3 root     root           100 Oct 24 03:46 .
drwxr-xr-x    2 root     root            60 Oct 24 03:46 ..2026_10_24_03_46_31.1043996967
lrwxrwxrwx    1 root     root            32 Oct 24 03:46 ..data → ..2026_10_24_03_46_31.1043996967
lrwxrwxrwx    1 root     root            12 Oct 24 03:45 token → ..data/token
```

No mesmo Pod, **o arquivo mudou para** `token-v2` **e a variável de ambiente ficou em** `token-v1`.



ANALISE

Allow x Forbid. Com um Job de 6 min agendado a cada 5 min, `Allow` sobrepõe execuções: das 00:50 às 00:51 houve 2 Jobs simultâneos. Para uma sincronização de catálogo isso é perigoso, porque duas execuções escrevendo no mesmo destino podem gravar dados fora de ordem. `Forbid` garante no máximo 1 ativo, mas o efeito colateral depende de `startingDeadlineSeconds`: a execução das 00:55 foi atrasada 63 s e a das 01:00 foi perdida. Se o job sistematicamente dura mais que o intervalo, `Forbid` não resolve: ele só esconde que o agendamento está errado, perdendo uma execução a cada ciclo. A correção real é o intervalo ser maior que a duração. `Replace` seria a terceira opção, matando a execução antiga para começar a nova, o que só serve quando a execução mais recente sempre invalida a anterior.

Env x volume. A variável de ambiente é resolvida **uma vez**, quando o container é criado: o kubelet lê o Secret, monta o ambiente do processo e nunca mais toca nele, porque o ambiente de um processo Linux não muda de fora. O volume é diferente: o kubelet sincroniza periodicamente o conteúdo e faz a troca de forma atômica. Ele grava a nova versão num diretório com timestamp (`..2026_10_24_03_46_31...`) e troca o symlink

`..data` para apontar para ela. O arquivo `token` é um symlink para `..data/token`, então quem lê o arquivo nunca vê um conteúdo pela metade. O tempo medido entre o patch (03:45:13Z) e a troca (03:46:31Z) foi de **78 s**, um pouco acima dos 60 a 70 s da dica. Por isso a checagem de 75 s do script ainda viu `token-v1`. O atraso vem do período de sincronização do kubelet somado ao cache de Secrets, e não é garantido. Consequências práticas: para rotação de credenciais sem restart, só volume funciona, e a aplicação ainda precisa **reler o arquivo** (ler uma vez no startup equivale a usar env). Com `subPath`, o volume também deixa de atualizar, porque o arquivo é montado direto e não pelo symlink.

Critérios de aceite

Critério	Resultado
`Allow`: 2 ou mais Jobs simultaneamente ativos	ok (2, das 00:50:00 às 00:51:00)
`Forbid`: no máximo 1 ativo e evento `JobAlreadyActive` no `describe`	ok
Histórico nunca acima de 2 Jobs concluídos	ok (máximo 2 em 47 amostras)
Arquivo atualizado e env desatualizada no mesmo Pod	ok (`token-v2` x `token-v1`)